A Guide to the Instant Insanity Starter Notebook

Qiskit Fall Fest Connecticut 2026 — qBraid challenge

Study notes

These notes walk through `starter_notebook.ipynb` section by section. Each part shows the code from the notebook and explains what it does and why. The numbers quoted come from the notebook's saved outputs. The notes cover only the code the organizers provided; the parts left for your team (`submit_job` and the core challenge) are described but not solved.

Contents

1 Setup **3**

2 The puzzle (Section 1) **3**

3 Brute force (Section 2) **3**
- 3.1 All orientations of one cube . . . 3
- 3.2 Checking a tower . . . 4
- 3.3 Trying every tower . . . 4

4 The graph trick (Section 3) **4**
- 4.1 The idea . . . 4
- 4.2 The code . . . 5
- 4.3 From a 2-factor to a tower . . . 5

5 A cost function (Section 4) **6**
- 5.1 Variables . . . 6
- 5.2 The cost . . . 6
- 5.3 Checking the cost classically . . . 7

6 From cost to qubits (Section 5) **7**
- 6.1 From bits to Pauli Z . . . 7
- 6.2 The code . . . 7
- 6.3 Checking H against the cost . . . 8

7 A first quantum run (Section 6) **9**
- 7.1 QAOA in one paragraph . . . 9
- 7.2 Building blocks . . . 9
- 7.3 Textbook QAOA . . . 10
- 7.4 Constrained QAOA: W states and the XY mixer . . . 10
- 7.5 Choosing the angles . . . 11
- 7.6 Running with shots . . . 12

8 Your challenge and the bonus generator (Section 7) 13
8.1 Making N-cube puzzles 13
8.2 Counting answers 13
8.3 What is left for you 14

A Quick reference 14

1 Setup

The first cell installs any missing packages. It uses `importlib.util.find_spec` to test whether each module can be imported, and only installs the ones that cannot. It does not upgrade packages that are already loaded, because replacing a package in a running kernel can break it.

```
NEEDED = {"qiskit": "qiskit>=1.3,<3", "qiskit_aer": "qiskit-aer>=0.15",
          "qbraid": "qbraid[qiskit]>=0.12", ...}
missing = [spec for mod, spec in NEEDED.items()
           if importlib.util.find_spec(mod) is None]
if missing:
    subprocess.run([sys.executable, "-m", "pip", "install", "--quiet", *missing],
                   check=True)
```

The second cell imports the basics and the fixed random seed:

```
import itertools
import time
import numpy as np
from resources.helpers import SEED      # SEED = 7
```

`SEED` is used for every random choice in the notebook, so each run gives the same numbers. The challenge rules require fixed seeds for reproducibility.

2 The puzzle (Section 1)

```
COLOURS = "RGBY"

CUBES = [
    ["GR", "BR", "GY"],   # cube 1
    ["GY", "RB", "BY"],   # cube 2
    ["BR", "RG", "RY"],   # cube 3
    ["YR", "RG", "BG"],   # cube 4
]
```

The data format. A cube has six faces, which come in three pairs of opposite faces. The notebook stores a cube as those three pairs. The string `"GR"` means a green face sits opposite a red face. The order of the two letters does not matter for the puzzle.

This is all the information the puzzle needs. Which face is "up" does not matter, only which colours sit opposite each other, because a tower only shows the front, back, left and right faces of each cube.

`draw_nets(CUBES)` draws each cube unfolded as a cross of six squares. Cube 3 has three red faces, which makes red the hardest colour to place.

3 Brute force (Section 2)

3.1 All orientations of one cube

```
def orientations(cube):
    """The 24 ways a cube can sit, as (front, back, left, right) colours."""
```

```
    views = []
    for a, b in itertools.permutations(range(3), 2):   # front/back pair, left/right
    pair
        for fa in (0, 1):
            for fb in (0, 1):
                fb_pair, lr_pair = cube[a], cube[b]
                views.append((fb_pair[fa], fb_pair[1 - fa], lr_pair[fb], lr_pair[1 - fb
    ]))
    return views
```

How the 24 is counted.

- `permutations(range(3), 2)` picks an *ordered* pair (a, b) of different pair indices: pair a goes on the front/back axis and pair b on the left/right axis. There are $3 \times 2 = 6$ such choices. The third pair is hidden on top and bottom.
- `fa` decides which colour of pair a faces front (the other faces back). `fb` does the same for left/right. That is $2 \times 2 = 4$ more choices.

In total $6 \times 4 = 24$, which matches the 24 rotations of a cube. Each view is a tuple `(front, back, left, right)`.

3.2 Checking a tower

```
def solves(tower):
    """True when each of the four sides shows four different colours."""
    return all(len({view[side] for view in tower}) == len(tower) for side in range(4))
```

A tower is a list of views, one per cube. For each of the four sides, the code collects that side's colours into a Python `set`. A set drops repeats, so its size equals the number of cubes only when no colour repeats. This works for any number of cubes, which matters for the bonus.

3.3 Trying every tower

```
options = [orientations(cube) for cube in CUBES]
winners = [tower for tower in itertools.product(*options) if solves(tower)]
```

`itertools.product` goes through every combination of one view per cube: $24^4 = 331{,}776$ towers. The saved output is *checked 331,776 towers in 0.4 s, found 8 solutions.* The 8 solutions are one tower seen in 8 ways: 4 quarter turns $\times$ 2 (upside down).

Takeaway: for 4 cubes, brute force takes under a second. Any quantum result for 4 cubes must be compared honestly against this.

4 The graph trick (Section 3)

4.1 The idea

Draw one dot per colour and one line per opposite pair. Each cube adds three lines (12 in total). Now look at only the front and back of a tower:

- each cube puts one of its pairs on the front/back axis, so we *pick one line per cube*;

- every colour must show once on the front and once on the back, so every colour dot must be touched by *exactly two* picked lines.

A set of lines that touches every dot exactly twice is called a **2-factor**. The left and right need a second 2-factor, and each cube must use a *different* line for it (a pair cannot face two directions at once). So each cube has $3 \times 2 = 6$ choices, and the search shrinks from 331,776 towers to $6^4 = 1{,}296$ cases.

4.2 The code

```
CHOICES = list(itertools.permutations(range(3), 2))  # (front/back pair, left/right
    pair)


def degrees(cubes, picks):
    """How many picked lines touch each colour."""
    count = dict.fromkeys(COLOURS, 0)
    for cube, p in zip(cubes, picks):
        for colour in cube[p]:
            count[colour] += 1
    return count


def is_two_factor(cubes, picks):
    return all(d == 2 for d in degrees(cubes, picks).values())


front_back = [picks for picks in itertools.product(range(3), repeat=len(CUBES))
              if is_two_factor(CUBES, picks)]
```

- `CHOICES` is the list of the 6 valid choices per cube, as `(front/back pair, left/right pair)`. It is used later for the full puzzle.
- `picks` holds one pair index per cube, e.g. `(0, 2, 1, 1)`.
- `degrees` counts how many times each colour appears among the picked pairs. In graph terms this is each dot's *degree*. A pair like `"RR"` would add 2 to red, which is correct: it is a loop.
- `is_two_factor` checks that every degree is exactly 2.
- `front_back` tries all $3^4 = 81$ picks. The output is *4 of 81 front/back picks are 2-factors.*

4.3 From a 2-factor to a tower

```
from resources.helpers import draw_tower, orient, tower_from

ends = orient(CUBES, front_back[0])
draw_tower([(ends[c][0], None, ends[c][1], None) for c in range(len(CUBES))])
```

A 2-factor splits into closed loops. `orient` walks each loop and gives each cube a direction: the colour where the walk enters becomes one side and the colour where it leaves becomes the

opposite side. Because a walk enters and leaves every dot once, each colour shows once on each side. `draw_tower` draws the result with left and right left blank (`None`).

`tower_from(cubes, choice)` does the same for a full answer (one `CHOICES` entry per cube) and returns a tower you can check with `solves`. Not all 4 front/back 2-factors can be completed: the matching left/right 2-factor must avoid each cube's front/back line.

5 A cost function (Section 4)

5.1 Variables

To hand the puzzle to an optimizer, every choice becomes a 0/1 variable:

$$x_{c,p} = 1 \text{ if cube } c \text{ uses pair } p \text{ on front/back}, \qquad y_{c,p} = 1 \text{ if cube } c \text{ uses pair } p \text{ on left/right}.$$

That is 6 variables per cube, 24 for four cubes. This is called a **one-hot** encoding, because a valid choice has exactly one 1 in each group of three.

5.2 The cost

$$C(x,y) = \lambda \sum_c \left[\left(\textstyle\sum_p x_{c,p} - 1\right)^2 + \left(\textstyle\sum_p y_{c,p} - 1\right)^2 \right] + \mu \textstyle\sum_{c,p} x_{c,p}\, y_{c,p} + \sum_{z\in\{x,y\}} \sum_k \left(d_k(z) - 2\right)^2$$

where $d_k(z)$ is the degree of colour k for the picks in z. Each term is a square or a product of 0/1 values, so it is never negative, and it is zero only when its rule is satisfied:

Term	Rule it enforces
$\lambda(\sum_p x_{c,p} - 1)^2$	cube c picks exactly one front/back pair
$\lambda(\sum_p y_{c,p} - 1)^2$	cube c picks exactly one left/right pair
$\mu\, x_{c,p} y_{c,p}$	a pair cannot serve both axes
$(d_k - 2)^2$	colour k is touched exactly twice (2-factor)

So $C = 0$ exactly when every rule holds, i.e. for a solved tower. A cost made of squared and pairwise terms in 0/1 variables is called a **QUBO** (quadratic unconstrained binary optimization). The weights λ and μ (`LAMBDA = MU = 2`) set how strongly each rule is punished.

```
LAMBDA, MU = 2, 2


def cost(cubes, x, y):
    """x[c][p], y[c][p] are 0/1. Zero exactly for a solved tower."""
    total = 0
    for c in range(len(cubes)):
        total += LAMBDA * ((sum(x[c]) - 1) ** 2 + (sum(y[c]) - 1) ** 2)
        total += MU * sum(x[c][p] * y[c][p] for p in range(3))
    for picks in (x, y):
        for k in COLOURS:
            d = sum(cubes[c][p].count(k) * picks[c][p]
                    for c in range(len(cubes)) for p in range(3))
            total += (d - 2) ** 2
    return total
```

```
def one_hot(p):
    return [int(i == p) for i in range(3)]
```

The code is a line-by-line copy of the formula. `cubes[c][p].count(k)` is how many times colour k appears in that pair (0, 1 or 2), so `d` is the degree d_k. `one_hot(1)` returns `[0, 1, 0]`. A random candidate scores 8 in the saved output.

5.3 Checking the cost classically

```
scores = []
for fb in itertools.product(range(3), repeat=len(CUBES)):
    for lr in itertools.product(range(3), repeat=len(CUBES)):
        scores.append(cost(CUBES, [one_hot(p) for p in fb], [one_hot(p) for p in lr]))
```

This scores every candidate with one pair per axis per cube: $3^4 \times 3^4 = 6{,}561$ of them. The output is *6,561 candidates, 2 at cost 0, lowest nonzero cost 2.* The two zeros are the answer and its quarter turn (front/back and left/right swapped).

Why this matters: if a wrong tower also scored zero, every optimizer, quantum or classical, would happily return it. The challenge rules ask you to run this kind of check on any new encoding *before* any quantum run.

6 From cost to qubits (Section 5)

6.1 From bits to Pauli Z

A qubit measured in state $|0\rangle$ or $|1\rangle$ gives one bit. The Pauli Z operator has value $+1$ on $|0\rangle$ and -1 on $|1\rangle$, so

$$x = \frac{1 - Z}{2} \quad \text{gives} \quad x = 0 \text{ on } |0\rangle,\ x = 1 \text{ on } |1\rangle.$$

Replace every variable in C by this expression and the cost becomes an operator H (a **Hamiltonian**). Its lowest-energy state (the **ground state**) is the solved tower. Because C is at most quadratic, H only contains single Z terms and ZZ pairs.

6.2 The code

```
from qiskit.quantum_info import SparsePauliOp


def build_hamiltonian(cubes, lam=LAMBDA, mu=MU):
    n = 6 * len(cubes)
    one = SparsePauliOp("I" * n)

    def var(q):  # the 0/1 variable on qubit q, as (1 - Z) / 2
        return SparsePauliOp.from_sparse_list([("", [], 0.5), ("Z", [q], -0.5)], n)

    xs = [[var(6 * c + p) for p in range(3)] for c in range(len(cubes))]
    ys = [[var(6 * c + 3 + p) for p in range(3)] for c in range(len(cubes))]
    H = 0 * one
    for c in range(len(cubes)):
        for row in (xs[c], ys[c]):
```

```
            pick = sum(row, -1 * one)
            H += lam * (pick @ pick)
        H += mu * sum(xs[c][p] @ ys[c][p] for p in range(3))
    for picks in (xs, ys):
        for k in COLOURS:
            d = -2 * one
            for c, cube in enumerate(cubes):
                for p in range(3):
                    if cube[p].count(k):
                        d += cube[p].count(k) * picks[c][p]
            H += d @ d
    return H.simplify()
```

- `SparsePauliOp` stores an operator as a sum of Pauli strings with coefficients. `one` is the identity on all n qubits.
- `var(q)` builds $(1 - Z_q)/2$: a constant 0.5 (the empty Pauli string) plus $-0.5\, Z_q$.
- **Qubit layout:** cube c uses qubits $6c$ to $6c + 5$. Qubits $6c + p$ hold $x_{c,p}$ (front/back) and qubits $6c + 3 + p$ hold $y_{c,p}$ (left/right).
- `@` multiplies operators, so `pick @ pick` is the square $(\sum_p x_{c,p} - 1)^2$. `sum(row, -1 * one)` starts the sum at -1.
- Each block mirrors one term of the cost function in Section 4.
- `simplify()` merges equal Pauli strings and drops zeros.

Output: *24 qubits, 153 Pauli terms, largest term size: 2* (so at most ZZ, as expected).

6.3 Checking H against the cost

```
def energy(H, bits):
    """Value of a Z-only Hamiltonian on a bitstring; bits[q] is qubit q."""
    total = 0.0
    for label, coeff in zip(H.paulis.to_labels(), H.coeffs):
        sign = 1
        for q, op in enumerate(reversed(label)):  # Qiskit labels put qubit 0 last
            if op == "Z" and bits[q]:
                sign = -sign
        total += coeff.real * sign
    return total
```

A Hamiltonian made only of Z operators is **diagonal**: on a single bitstring its value is plain arithmetic. Each Z_q contributes -1 if bit q is 1 and $+1$ otherwise, so a term's sign flips once per Z on a 1-bit.

Bit order: Qiskit writes Pauli labels (and measurement results) with qubit 0 as the *rightmost* character. That is why the loop uses `reversed(label)`.

The next cell evaluates H and `cost` on 200 random bitstrings, including invalid ones, and reports the largest difference: *0.0e+00*. So the Hamiltonian is exactly the puzzle.

Size. 24 qubits means $2^{24} \approx 16.8$ million amplitudes. SV1 handles that, a laptop slows down, and only 2 of those bitstrings are answers. Section 6 therefore works on a smaller version first.

7 A first quantum run (Section 6)

Section 6 solves only the *front/back* half with QAOA on 12 qubits (3 per cube, qubit $3c+p$ is $x_{c,p}$).

7.1 QAOA in one paragraph

QAOA (Quantum Approximate Optimization Algorithm) prepares a starting state, then repeats p rounds of two layers:

$$|\psi(\gamma,\beta)\rangle = U_M(\beta_p)\,U_C(\gamma_p)\cdots U_M(\beta_1)\,U_C(\gamma_1)\,|\psi_0\rangle, \qquad U_C(\gamma)=e^{-i\gamma H}, \quad U_M(\beta)=e^{-i\beta M}.$$

The **cost layer** U_C gives each bitstring a phase that depends on its energy. The **mixer** U_M moves amplitude between bitstrings so those phases turn into changes of probability. A classical optimizer tunes the angles γ, β to make low-energy bitstrings likely. Measuring then samples answers.

7.2 Building blocks

```
def cost_layer(qc, H, gamma):
    """exp(-i gamma H) for a Hamiltonian of Z and ZZ terms."""
    for label, coeff in zip(H.paulis.to_labels(), H.coeffs):
        qubits = [q for q, op in enumerate(reversed(label)) if op == "Z"]
        if len(qubits) == 1:
            qc.rz(2 * gamma * coeff.real, qubits[0])
        elif len(qubits) == 2:
            qc.rzz(2 * gamma * coeff.real, *qubits)
```

All terms of H commute (they are all Z's), so $e^{-i\gamma H}$ is a product of one small gate per term. Qiskit defines $R_Z(\theta) = e^{-i\theta Z/2}$ and $R_{ZZ}(\theta) = e^{-i\theta ZZ/2}$, so the term $c\,Z$ needs angle $\theta = 2\gamma c$. That is where the factor 2 comes from. The identity term only adds a global phase, which cannot be measured, so it is skipped.

```
def front_back_hamiltonian(cubes, lam=0):
    """Degree rule on the x variables only; lam adds the one-pair-per-cube penalty."""
    ...
    for k in COLOURS:
        d = -2 * one
        for c, cube in enumerate(cubes):
            for p in range(3):
                d += cube[p].count(k) * var(3 * c + p)
        H += d @ d
    for c in range(len(cubes)):
        pick = sum((var(3 * c + p) for p in range(3)), -1 * one)
        H += lam * (pick @ pick)
    return H.simplify()


GOOD = solution_strings(front_back)      # 4 good bitstrings
```

This is the 12-qubit version: only the 2-factor rule on the x variables. With `lam=0` there is no one-pair-per-cube penalty, which is fine when the circuit itself never leaves the valid choices (Section 7.4). `GOOD` turns the 4 front/back 2-factors into the bitstrings a measurement would return, in Qiskit order.

7.3 Textbook QAOA

```
H_pen = front_back_hamiltonian(CUBES, lam=2)
plain = QuantumCircuit(N_Q)
plain.h(range(N_Q))              # |+> on every qubit
cost_layer(plain, H_pen, gamma)
plain.rx(2 * beta, range(N_Q))   # standard X mixer
```

- Hadamards put every qubit in $|+\rangle$: an equal spread over all $2^{12} = 4{,}096$ bitstrings.
- $R_X(2\beta) = e^{-i\beta X}$ on every qubit is the standard mixer.
- Most of the 4,096 bitstrings pick 0, 2 or 3 pairs for some cube, so the Hamiltonian needs the penalty (`lam=2`) to push them up.

The helper `scan` tries a 25×13 grid of (γ, β) at $p = 1$:

```
def scan(template, H, gammas, betas):
    costs, wins = diagonal(H), [int(s, 2) for s in GOOD]
    ...
            probs = Statevector(template.assign_parameters({gamma: g, beta: b})).
    probabilities()
            avg[i, j], hit[i, j] = probs @ costs, probs[wins].sum()
```

`Statevector` computes the exact quantum state, so there are no shots and no sampling noise. `diagonal(H)` lists the energy of every bitstring, so `probs @ costs` is the *average* energy $\langle H \rangle$ and `probs[wins].sum()` is the chance of measuring a good bitstring.

Result: *P(good pick) 1.38% (random guess 0.10%).* That is about 14 times better than guessing, but still wrong 99 times in 100: most amplitude sits on invalid bitstrings.

7.4 Constrained QAOA: W states and the XY mixer

Idea: never leave the valid choices. Start each cube in the **W state**

$$|W\rangle = \tfrac{1}{\sqrt{3}}\big(|001\rangle + |010\rangle + |100\rangle\big),$$

an equal spread over that cube's 3 one-hot choices.

```
def w_state(qc, a, b, c):
    """Equal spread over |100>, |010>, |001> on qubits a, b, c."""
    qc.ry(2 * np.arcsin(1 / np.sqrt(3)), a)
    qc.append(RYGate(np.pi / 2).control(1, ctrl_state=0), [a, b])
    qc.append(XGate().control(2, ctrl_state="00"), [a, b, c])
```

1. $R_Y(\theta)|0\rangle = \cos\frac{\theta}{2}|0\rangle + \sin\frac{\theta}{2}|1\rangle$. With $\sin\frac{\theta}{2} = 1/\sqrt{3}$, qubit a is 1 with probability $1/3$.

2. If a is 0 (`ctrl_state=0` means "control on 0"), rotate b by $\pi/2$: it becomes 1 with probability $1/2$ of the remaining $2/3$, i.e. $1/3$.

3. If both a and b are 0, flip c to 1 (the last $1/3$).

Each of the three one-hot strings now has probability $1/3$.

```
def xy_mixer(qc, beta, cubes):
    for c in range(len(cubes)):
        q = [3 * c, 3 * c + 1, 3 * c + 2]
        for i, j in ((0, 1), (0, 2), (1, 2)):
            qc.append(XXPlusYYGate(2 * beta), [q[i], q[j]])
```

The `XXPlusYYGate` mixes $|01\rangle$ and $|10\rangle$ on two qubits and leaves $|00\rangle$ and $|11\rangle$ alone. It moves a 1 from one qubit to another without creating or removing one, so the number of 1s per cube stays exactly one. Applied to each pair of a cube's qubits, it lets amplitude flow between all three choices.

```
def constrained_qaoa(H, gammas, betas, cubes=CUBES):
    qc = QuantumCircuit(3 * len(cubes))
    for c in range(len(cubes)):
        w_state(qc, 3 * c, 3 * c + 1, 3 * c + 2)
    for g, b in zip(gammas, betas):
        qc.barrier(label="cost")
        cost_layer(qc, H, g)
        qc.barrier(label="mixer")
        xy_mixer(qc, b, cubes)
    return qc
```

One W state per cube, then p rounds of cost layer and XY mixer. The circuit now only explores the $3^4 = 81$ valid picks, so a random valid pick succeeds with probability $4/81 \approx 4.9\%$. Barriers only label the drawing.

7.5 Choosing the angles

Grid scan, $p = 1$.

```
xy = constrained_qaoa(H_fb, [gamma], [beta])
gammas, betas = np.linspace(0, np.pi, 41), np.linspace(0, np.pi, 41)
avg, hit = scan(xy, H_fb, gammas, betas)
```

Best angles $\gamma \approx 2.827$, $\beta \approx 0.236$; *P(good pick) 18.8% (random valid pick 4.9%).* The heatmap has several valleys. A local optimizer started in the wrong valley gets stuck there, so a coarse scan comes first.

COBYLA, $p = 2$.

```
def average_cost(angles):
    p = len(angles) // 2
    probs = Statevector(constrained_qaoa(H_fb, angles[:p], angles[p:])).probabilities()
    history.append(probs @ costs)
    return history[-1]


result = minimize(average_cost, [g1, g1, b1, b1], method="COBYLA", options={"maxiter":
    200})
```

`angles` holds $[\gamma_1, \gamma_2, \beta_1, \beta_2]$. COBYLA is a SciPy optimizer that needs no gradients. It starts from the $p = 1$ angles repeated. Result: *p=2 after 125 evaluations: average cost 1.56, P(good pick) 28.4%.*

This is the **hybrid loop** of QAOA: the quantum part prepares the state and produces samples; the classical part (COBYLA) chooses the angles.

7.6 Running with shots

```
def run_on_qbraid(circuit, shots=1000):
    """Prepare the circuit, send it with your submit_job, return counts in Qiskit order
    ."""
    qc = prepare(circuit)
    raw = submit_job(qc, shots)
    if raw is None:
        print("submit_job returned nothing yet, so this ran on the local Aer simulator.
    ")
        raw = AerSimulator(seed_simulator=SEED).run(qc, shots=shots).result().
    get_counts()
    return strip_marker(raw)
```

- `prepare` (helper) adds a *marker qubit* fixed at 1, measures every qubit, and transpiles to the basic gates `["rz", "sx", "x", "cx"]`. Custom gates like the W-state and XY gates are not available on every device. The marker shows which end of the result string is which, because providers do not all order bits the same way.
- `strip_marker` removes the marker and returns counts in Qiskit order (qubit 0 rightmost).
- If `submit_job` returns `None`, the circuit runs on the local `AerSimulator` instead, for free.

`submit_job`: your part.

```
SV1 = "aws:aws:sim:sv1"


def submit_job(qc, shots):
    """Your code: run qc on a qBraid device and return its counts dictionary."""
    # 1. create a provider for your qBraid account
    # 2. get the device with id SV1 and print its status
    # 3. run qc with this many shots, print the job id, and return the counts
    return None
```

This stub is left for your team. The three comments are the steps. `resources/02_qbraid_sdk.ipynb` and the qBraid provider guide show the API you need. Remember to print the job ID; it goes in your results table. Running on SV1 costs credits.

The result.

```
counts = run_on_qbraid(constrained_qaoa(H_fb, best[:2], best[2:]), shots=shots)
good = sum(counts.get(s, 0) for s in GOOD)
```

With the local fallback: *282 of 1000 shots are good picks (28.2%); exact value 28.4%.* With shots, the measured fraction wobbles around the exact probability.

Reading a result string: read right to left in groups of three. The first (rightmost) group is cube 1, and the position of its 1 is the chosen pair.

8 Your challenge and the bonus generator (Section 7)

8.1 Making N-cube puzzles

```
def make_puzzle(n, seed):
    """N cubes, N colours, a planted answer: two random cycles through all colours."""
    rng = random.Random(seed)
    letters = "".join(chr(ord("A") + i) for i in range(n))
    cycles = []
    for _ in range(2):
        order = rng.sample(letters, n)
        lines = [order[i] + order[(i + 1) % n] for i in range(n)]
        rng.shuffle(lines)
        cycles.append(lines)
    cubes = []
    for i in range(n):
        third = rng.choice(letters) + rng.choice(letters)
        pairs = ["".join(rng.sample(pair, 2)) for pair in (cycles[0][i], cycles[1][i],
    third)]
        rng.shuffle(pairs)
        cubes.append(pairs)
    return cubes
```

- Colours are named `A`, `B`, ...
- It builds two random loops through all n colours. A single loop through every dot is a 2-factor, so these two loops are a *planted* answer.
- Cube i gets one line from each loop plus a random third pair, with the letters and pairs shuffled so the answer is hidden.

Every generated puzzle has at least one answer, but it may have more.

8.2 Counting answers

```
def count_answers(cubes):
    """How many of the 6**n cases solve the puzzle."""
    letters = sorted({ch for cube in cubes for pair in cube for ch in pair})
    total = 0
    for choice in itertools.product(CHOICES, repeat=len(cubes)):
        for side in (0, 1):
            deg = dict.fromkeys(letters, 0)
            for cube, pick in zip(cubes, choice):
                for ch in cube[pick[side]]:
                    deg[ch] += 1
            if any(d != 2 for d in deg.values()):
                break
        else:
            total += 1
    return total
```

This is the graph method from Section 3, generalized: try all 6^n choices and check that both the front/back picks (`side = 0`) and the left/right picks (`side = 1`) are 2-factors. The `for ... else` adds to `total` only if the loop did not `break`, i.e. both sides passed.

```
BONUS = {5: 16, 6: 86, 7: 68, 8: 4}
```

These seeds give puzzles with exactly one tower (2 cases: the tower and its quarter turn). Saved output:

N	cases (6^N)	answers	time	one-hot qubits ($6N$)
5	7,776	2	0.0 s	30
6	46,656	2	0.1 s	36
7	279,936	2	0.6 s	42
8	1,679,616	2	4.1 s	48

The classical search grows by 6× per cube, and the one-hot encoding needs 6 qubits per cube. SV1 simulates at most 34 qubits, so the one-hot encoding cannot go past $N = 5$ there. This is why the notebook suggests looking for smaller encodings.

8.3 What is left for you

- **Core:** add left/right and find a full solved tower with a quantum algorithm, run it on SV1, and report success rate, qubits, two-qubit gates, depth, credits, and the random-guess baseline.
- **Bonus:** generalize to N cubes and colours, using `make_puzzle` and `count_answers`.

A Quick reference

Name	What it does
`CUBES`, `COLOURS`	The four-cube puzzle and the colour letters
`orientations`, `solves`	The 24 views of a cube; the four-sides check
`CHOICES`	The 6 valid (front/back, left/right) choices per cube
`degrees`, `is_two_factor`	Colour degrees of a pick; the 2-factor test
`front_back`	The 4 front/back 2-factors
`orient`, `tower_from`	Turn picks into front/back colours or a full tower
`cost`	The QUBO cost, zero only for a solved tower
`build_hamiltonian`	24-qubit Hamiltonian (qubit $6c + p = x$, $6c + 3 + p = y$)
`front_back_hamiltonian`	12-qubit warm-up Hamiltonian (qubit $3c + p$)
`energy`, `diagonal`	Value of a Z-only H on one bitstring / on all
`cost_layer`	$e^{-i\gamma H}$ from `rz` and `rzz` gates
`w_state`, `xy_mixer`	One-hot-preserving start state and mixer
`constrained_qaoa`	The warm-up QAOA circuit
`prepare`, `strip_marker`	Transpile with a marker qubit; remove it after
`run_on_qbraid`, `submit_job`	Run a circuit; `submit_job` is yours to write
`make_puzzle`, `count_answers`	Bonus puzzle generator and brute-force counter

Qiskit bit order. In Pauli labels and measurement count keys, qubit 0 is the *rightmost* character. Test any decoder on a known answer before trusting it.